Test-Driven Development

App Inventra (Android, Kotlin + Jetpack Compose)

Ticket: INV2-153 · épico INV2-10
Repositório: InventraTech/inventra-mobile-development-2, branch `feat/estado-rotacao`
Ferramentas: JUnit 4, kotlinx.serialization, Kover 0.9.11
Autor: Felipe Kogake
Data: 25/10/2026

1. Funcionalidade escolhida

A pilha de telas sobrevive quando o celular gira.

Antes, o `Navigator` era criado com `remember`. Ao girar a tela, o Android recria a Activity, e o `remember` perde o que guardava. Quem estava, por exemplo, no detalhe de um produto voltava para a tela inicial.

A correção foi um `Saver`, que transforma o `Navigator` em texto (JSON) para o Android guardar e depois o remonta. Ele foi escolhido para o TDD por três motivos:

- **É funcionalidade nova:** o código não existia, então dava para escrever o teste antes.
- **É lógica pura:** roda na JVM, sem emulador.
- **Tem casos de borda reais:** perder a tela inicial do comprador, estado salvo ilegível depois de uma atualização do app e pilha vazia.

2. Ciclos Red → Green → Refactor

Os testes estão em NavigatorSaverTest.kt e o código em NavigatorSaver.kt. Cada etapa é um commit separado, e cada hash abaixo é um link para o commit no GitHub: o histórico mostra o teste chegando antes do código.

Ciclo 1: a pilha volta igual

Etapa	Commit	O que aconteceu
Red	`0c2ef62`	Teste *pilha de telas volta igual depois de girar a tela* com a pilha `[Home, StockList, StockDetail(7)]`. O saver era só um esqueleto com `TODO()`. Falha: `kotlin.NotImplementedError`.
Green	`c83fb02`	`@Serializable` em todas as telas do `Screen` e o saver grava e lê a lista em JSON. 1 de 1 teste passando.
Refactor	`52d67f8`	O tipo `List<Screen>`, que se repetia, virou um único `ListSerializer(Screen.serializer())`. O teste continua verde.

Ciclo 2: a tela inicial do perfil não se perde

Etapa	Commit	O que aconteceu
Red	`e276fd2`	Teste *comprador continua com as requisicoes como inicio depois de girar a tela*. O comprador tem `RequirementsList` como início, não `Home`. Falha: `expected:<RequirementsList> but was:<Home>`.
Green	`2e5f422`	O estado salvo virou `SavedNavigator(backStack, home)`, guardando também a tela inicial. 2 de 2 passando.
Refactor	`5df9169`	A conversão entre `Navigator` e `SavedNavigator` foi para funções próprias: `toSaved()` e `toNavigator()`.

Ciclo 3: estado salvo ilegível não derruba o app

Etapa	Commit	O que aconteceu
Red	`769b057`	Teste *estado salvo ilegivel abre o app pela pilha inicial*. Simula um estado salvo com uma tela que não existe mais, o que acontece se uma atualização do app renomear uma tela. Falha: `JsonDecodingException`. No celular, isso seria o app fechando sozinho.
Green	`37eaceb`	A leitura passou a usar `runCatching`. Se não der para ler, o saver devolve `null`, e o `rememberSaveable` usa a pilha inicial. 3 de 3 passando.
Refactor	não houve	O código já estava simples; não foi forçado um commit só para cumprir a etapa.

Ciclo 4: pilha vazia não é restaurada

Etapa	Commit	O que aconteceu
Red	`dd10be6`	Teste *pilha salva vazia abre o app pela pilha inicial*. Falha no `assertNull`: o saver devolvia um `Navigator` sem telas. Na tela, o `NavDisplay` quebraria ao buscar a última tela de uma pilha vazia.
Green	`aa08e27`	`takeIf { it.backStack.isNotEmpty() }` descarta a pilha vazia. 4 de 4 passando.
Refactor	`32c5087`	Salvar e restaurar viraram funções nomeadas (`encode()` e `decodeNavigator()`), e o `navigatorSaver()` ficou com duas linhas.

Código final

```
@Serializable
private data class SavedNavigator(
    val backStack: List<Screen>,
    val home: Screen,
) {
    fun toNavigator() = Navigator(backStack.toMutableStateList(), home)
}

private fun Navigator.toSaved() = SavedNavigator(currentBackStack, home)

private fun Navigator.encode(): String = Json.encodeToString(SavedNavigator.serializer(), toSaved())

private fun decodeNavigator(saved: String): Navigator? =
    runCatching { Json.decodeFromString(SavedNavigator.serializer(), saved) }
        .getOrNull()
        ?.takeIf { it.backStack.isNotEmpty() }
        ?.toNavigator()

fun navigatorSaver(): Saver<Navigator, String> =
    Saver(
        save = { it.encode() },
        restore = ::decodeNavigator,
    )
```

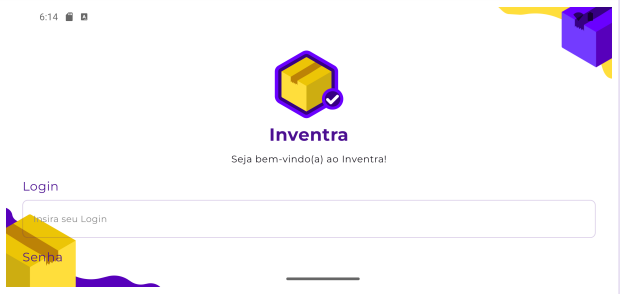
3. Ligação na tela

O commit `2451e22` troca `remember` por `rememberSaveable(saver = navigatorSaver())` em SetupNavagtion.kt. Essa parte é código Compose, que só roda com tela, então foi validada no emulador (Pixel 7 Pro, Android 14) em vez de teste unitário:

1. No Login, toquei em "Criar conta".
2. Girei o emulador para paisagem: o app continuou no Cadastro.
3. Apertei Voltar: o app foi para o Login, o que mostra que a pilha inteira voltou, não só a tela do topo.



Cadastro depois de girar



Voltar depois de girar: Login

4. Cobertura de testes

Medida com o Kover, adicionado no commit `989daf1`, antes do primeiro teste. Funções `@Composable` e `@Preview` ficam fora da conta porque são telas, que não têm teste unitário. Contadas, elas só baixariam o número sem dizer nada sobre a lógica.

Escopo	Antes (989daf1)	Depois
App inteiro (linhas)	74,5% (2828/3796)	74,7% (2859/3826)
Pacote ui/navigation	53,6% (60/112)	64,1% (91/142)
NavigatorSaver.kt	não existia	**100% (9/9)**

Total de testes unitários: **435, todos passando**, além das verificações de estilo (`ktlintCheck` e `detekt`).

5. Como reproduzir

Rodar os testes do saver:

```
./gradlew testDebugUnitTest --tests "*NavigatorSaverTest*"
```

Gerar o relatório de cobertura em `app/build/reports/kover/htmlDebug/index.html`:

```
./gradlew koverHtmlReportDebug
```

Para ver um red de novo, basta ir ao commit do teste e rodar o mesmo comando, por exemplo `git checkout e276fd2`.